Sugi CMMS Learning Guide

A step-by-step, beginner-friendly explanation of the React PWA, API, SQLite database, uploads, and Work Order MVP we just built.

Read this like a tour. First understand the map, then open the files one by one.

1. What We Built

We built the foundation of a CMMS. CMMS means Computerized Maintenance Management System. In factory language, it is the system that helps people issue, receive, track, update, and close maintenance jobs.

The first real feature is Work Orders. Other big CMMS areas are already visible as placeholder pages, so the app feels like a real system from the beginning.

Part	What it does	Where it lives
Frontend	The screen users touch: dashboard, forms, buttons, technician view, TV board.	apps/web
Backend/API	The kitchen behind the counter. It receives requests, saves data, and sends answers.	apps/api
Shared Types	The common dictionary that tells both sides what a work order looks like.	packages/shared
Database	The notebook where work orders, users, activities, and notifications are stored.	apps/api/data/cmms.sqlite
Uploads	The folder where work order images are stored.	apps/api/uploads

The system map

```
User phone/laptop
  -> React PWA screen
  -> API client fetch()
  -> Express API route
  -> Database function
  -> SQLite database and uploads folder
  -> JSON response comes back
  -> React updates the screen
```

Explain like you are 5: the frontend is the shop counter, the backend is the staff-only kitchen, the database is the notebook, and JSON is the little paper note passed between the counter and kitchen.

2. Weird Terms, Made Friendly

Term	Simple meaning
React	A tool for building screens using small reusable pieces called components.
Component	A Lego block of the screen. A button, card, page, or layout can be a component.
TypeScript	JavaScript with labels. It helps catch mistakes before the app runs.
PWA	A website that can behave like an app: installable, has an icon, and can later support push/offline features.
API	A menu of actions the frontend can ask the backend to do.
Endpoint	One API address, like /api/work-orders.
Express	The backend web server library. It listens for API requests.
SQLite	A small database saved as a file. Good for learning and MVP.
Vite	The tool that starts and builds the React app quickly.
pnpm	The package installer. It downloads React, Express, Vite, and friends.
Monorepo	One project folder that contains frontend, backend, and shared code together.
Route	A URL path that shows a page, like /work-orders/new.
State	A component's memory. Example: search text, selected user, open notification panel.
Props	Information passed into a component. Like giving a card its title and status.
Service worker	A small browser helper for PWA behavior and offline shell caching.
Manifest	The PWA app label: name, icon, colors, and install behavior.

3. Step-by-Step: What I Did

#	Step	What happened
1	Created the workspace foundation.	Added package.json, pnpm-workspace.yaml, .gitignore, and README.md. This made the project a monorepo with scripts for dev, build, and typecheck.
2	Created shared TypeScript types.	Added User, WorkOrder, WorkOrderStatus, NotificationRecord, and other shared shapes in packages/shared/src/index.ts.
3	Created the backend API.	Added apps/api with Express routes for users, dashboard summary, work orders, comments, status changes, attachments, and notifications.
4	Created the SQLite database layer.	Used Node 24's built-in node:sqlite module. The API creates tables and seed data automatically when it starts.
5	Created upload storage.	Added multer for image uploads. Files go into apps/api/uploads, while database rows store file metadata and URL paths.
6	Created the React PWA app.	Added apps/web with Vite, React Router, manifest.webmanifest, sw.js, and app icon.
7	Created main pages.	Built dashboard, work order list, create form, detail page, technician queue, TV dashboard, and placeholders for future CMMS modules.
8	Connected frontend to backend.	Added apps/web/src/api/client.ts. This file uses fetch() to call the API and return typed data.
9	Added notification behavior.	The API creates notification records during work order events. The React topbar polls notifications every 15 seconds.
10	Verified the app.	Ran pnpm typecheck, pnpm build, tested API health, created a work order, moved it through the flow, and tested uploads.

4. Folder Tour

```
Sugi CMMS System
  apps
    api
      src
        server.ts       API routes live here
        db.ts           database, workflow, notifications
      data              SQLite file appears here
      uploads           uploaded work order images appear here
    web
      src
        main.tsx        React starts here
        App.tsx         page routes live here
        api/client.ts   frontend API calls live here
        components      reusable UI blocks
        pages           dashboard, work orders, technician, TV
      public
        manifest.webmanifest
        sw.js
  packages
    shared
      src/index.ts      shared TypeScript types
```

How to read the project first

1. Open packages/shared/src/index.ts to understand the data shapes.
2. Open apps/web/src/main.tsx to see how React starts.
3. Open apps/web/src/App.tsx to see all pages and routes.
4. Open apps/web/src/components/Layout.tsx to see navigation and notifications.
5. Open apps/api/src/server.ts to see the API menu.
6. Open apps/api/src/db.ts to see how work orders are saved and updated.

5. Work Order Flow

This flow follows your factory process: requester issues, maintenance acknowledges, technician works, requester verifies.

Status	Meaning	Who usually acts
open	Requester issued a new work order. Maintenance must notice it.	Requester
acknowledged	Maintenance saw it. If technician self-assigns, assignment happens here.	Technician
in_progress	Repair or fabrication has started.	Technician
pending_material	Maintenance is waiting for spare part, material, approval, or outside help.	Technician or executive
resolved	Maintenance says the work is done and requester should check.	Technician
closed	Requester verified the work and closes the job.	Requester
returned	Requester says not okay. Work goes back to maintenance.	Requester
cancelled	Work order is stopped and no longer active.	Executive or admin later

The simple flow picture

```
open
  -> acknowledged
  -> in_progress
  -> pending_material -> in_progress
  -> resolved
  -> closed

If requester is not happy:
resolved -> returned -> in_progress
```

6. Frontend: How React Holds the Screen

React builds the screen from components. A component is just a function that returns UI. You can think of it as a recipe for one piece of screen.

React starts here

```
main.tsx
  -> wraps the app in BrowserRouter
  -> wraps the app in UserProvider
  -> renders App
  -> registers the service worker
```

Routes decide the page

```
App.tsx
  /                    DashboardPage
  /work-orders         WorkOrdersPage
  /work-orders/new     CreateWorkOrderPage
  /work-orders/:id     WorkOrderDetailPage
  /technician          TechnicianPage
  /tv                  TvDashboardPage
```

Explain like you are 5: React Router is like a receptionist. You tell it the address, and it brings you to the correct room.

State is screen memory

Example: the work order list remembers your search text, selected status filter, and whether you want all work orders or only yours. That memory is called state.

```
const [search, setSearch] = useState("");
const [status, setStatus] = useState("all");
```

7. Backend: How the API Works

The backend is the rule keeper. The frontend should not directly touch the database. It asks the backend through API endpoints.

Endpoint	Purpose
GET /api/health	Check if backend is alive.
GET /api/users	Get seed users and roles.
GET /api/dashboard-summary	Get dashboard counts.
GET /api/work-orders	List work orders.
POST /api/work-orders	Create a new work order.
GET /api/work-orders/:id	Get one work order with timeline and images.
PATCH /api/work-orders/:id/status	Move a work order to another status.
PATCH /api/work-orders/:id/assign	Assign a technician.
POST /api/work-orders/:id/comments	Add a timeline comment.
POST /api/work-orders/:id/attachments	Upload images.
GET /api/notifications	Get user notifications.

What happens when you create a work order

```
1. React form collects title, description, asset, location, priority.
2. api.createWorkOrder() sends JSON to POST /api/work-orders.
3. Express route validates the input.
4. createWorkOrder() inserts a row into SQLite.
5. addActivity() writes "Work order issued" into the timeline.
6. notifyUsers() creates notifications for maintenance users.
7. API sends the new work order back to React.
8. React navigates to the work order detail page.
```

8. Database and Uploads

SQLite stores structured records. Upload files are different: images are saved in a folder, and the database stores the filename, URL, uploader, size, and type.

Table	What it stores
users	Seed users such as requester, technician, executive, admin.
work_orders	Main work order record.
work_order_activities	Timeline events: created, started, resolved, commented.
work_order_attachments	Metadata for uploaded images.
notifications	In-app notification records for each user.

Why images are not stored inside the database

- Images can be large; databases prefer small structured records.
- Files are easier to back up, move, and serve from a folder or cloud storage later.
- The database only needs to remember where the file is.

```
Image file:
  apps/api/uploads/work-orders/{workOrderId}/{filename}

Database row:
  workOrderId, uploadedBy, filename, originalName, mimeType, size, url, kind
```

9. PWA Foundation

The PWA foundation makes the web app feel more like an installed app on a phone. We added the app manifest, icon, and service worker.

File	Purpose
apps/web/public/manifest.webmanifest	App name, short name, theme color, icon, install behavior.
apps/web/public/sw.js	Service worker. It caches the app shell and helps with offline fallback later.
apps/web/src/pwa/registerServiceWorker.ts	Registers the service worker in the browser.
apps/web/public/icons/cmms-icon.svg	Installable app icon.

Important: push notification is not finished yet. We prepared the PWA base first. Web push can be added after authentication and notification settings are stable.

10. How to Run and Verify

```
pnpm install
pnpm dev
```

Thing	URL or command
Web app	http://localhost:5173
TV board	http://localhost:5173/tv
API health	http://localhost:4000/api/health
Type check	pnpm typecheck
Production build	pnpm build

11. Your Learning Exercises

Exercise	What to do	What you learn
1	Open App.tsx and add a fake route called /training.	How routing maps URL to page.
2	Open WorkOrdersPage.tsx and change the default filter from all to mine.	How state controls screen behavior.
3	Open DashboardPage.tsx and rename one metric label.	How components display data.
4	Create a new work order from the browser, then find it in SQLite later.	How frontend, API, and database connect.
5	Upload an image in a work order and check apps/api/uploads.	How file upload storage works.
6	Open Technician view as a technician user and acknowledge a work order.	How roles affect workflow actions.
7	Open TV mode and watch it update after changing a work order.	How one data source can power many screens.

Recommended reading order

1. README.md
2. packages/shared/src/index.ts
3. apps/web/src/main.tsx
4. apps/web/src/App.tsx
5. apps/web/src/pages/WorkOrdersPage.tsx
6. apps/web/src/pages/WorkOrderDetailPage.tsx
7. apps/web/src/api/client.ts
8. apps/api/src/server.ts
9. apps/api/src/db.ts

12. What We Should Build Next

- Real login instead of user switcher.
- Better role permissions and department access.
- Work order validation rules and required resolution fields.
- Web push notification after login is stable.
- Asset register with QR code.
- Spare part tracker and material requests.
- Preventive maintenance schedules.
- Reports and performance dashboard.

The foundation is intentionally simple and readable. The goal is not only to have a CMMS; it is also for you to understand how each piece works.